Project Proposal:
Travel Style Segmentation & Drift Monitoring

Course: ITCS355 — ML Operation and Deployment
Team Members:
1.Perawis Buranasing (6688012)
2. Supitsara Tanasarnsukstid (6688121)
3. Atichat Kangsamut (6688193)

Cloud Platform: GCP

1. Problem and Users

Travelers may like different types of places, such as beaches, museums, cafés, parks, or shopping malls. However, it can be difficult for them to clearly describe their travel style.

This project will build a machine learning service that groups users by their travel preferences. Possible groups may include Nature and Relaxation, Food and Café, Culture and Sightseeing, or Shopping and Entertainment. The final group names will be based on the patterns found in the data.

The main users are travelers, travel platforms, and tourism teams that want to understand different travel preferences. The groups are only preference patterns. They are not fixed personality labels and should not be used for high-impact decisions.

2. Dataset

We will use the **Travel Review Ratings** dataset from the UCI Machine Learning Repository.

- Dataset URL: https://archive.ics.uci.edu/dataset/485/tarvel%2Breview%2Bratings
- Size: 5,456 user profiles
- Features: Ratings for 24 types of places
- Examples: beaches, parks, museums, restaurants, cafés, hotels, shopping malls, viewpoints, and gardens
- License: Creative Commons Attribution 4.0 International (CC BY 4.0)

The user ID will be used only to create repeatable data splits. It will not be used as a model feature. The data and source code will be versioned with Git and DVC.

3. Machine Learning Approach

We will use a clustering model because the dataset does not contain fixed travel-style labels. K-Means will be the main model, with another clustering method used as a simple comparison.

We will test different numbers of groups and random seeds. We will select the model by considering cluster stability, Silhouette score, cluster size, distance to cluster centers, and how clearly each cluster can be explained.

All experiments will be recorded in MLflow. The selected model, scaler, feature order, cluster centers, and segment-name mapping will be stored together and registered with full lineage to the exact code and data versions.

4. Serving Pattern and Requirements

We will deploy the model as an **online FastAPI service**. A user will send ratings for the 24 place categories, and the API will return a travel segment.

- `POST /predict` for one profile
- `POST /predict/batch` for up to 100 profiles
- `GET /health` for service health
- `GET /ready` for model readiness

Our latency target is **p95 below 300 milliseconds at concurrency 10**, excluding serverless cold starts. The model will be loaded once when the service starts.

The input profile is fresh at request time. The model will not retrain on a fixed schedule without a reason. Retraining will be considered when a new data version is available or when monitoring shows stable and meaningful drift.

5. Monitoring and Alert

The dashboard will show request and error rates, p50/p95/p99 latency, the current model version, the number of users in each travel segment, average distance from users to cluster centers, unassigned-profile rate, input-feature drift, and invalid requests.

At least one real alert will be configured. The main alert will fire when the unassigned-profile rate or segment-distribution drift is higher than the threshold measured from the validation data.

6. Planned Failure

Our planned failure is an incorrect segment-name mapping after model retraining. Cluster numbers may change even when the cluster patterns are similar. For example, Cluster 0 may mean Nature in one model version but Food in the next version.

We will deliberately deploy an incorrect mapping to staging and replay known user profiles. The dashboard should show a sudden change in the segment distribution, and the alert should fire. We will then roll back the bad version, inspect the logs, and add a regression test that matches new cluster centers to the existing segment names before promotion.

Failure → Monitoring → Alert → Investigation → Rollback → New CI Test

7. CI/CD and Testing

GitHub Actions will run unit tests, data-contract tests, model-behavior tests, cluster-mapping tests, a container build, and an API integration test.

A bad commit that changes a required feature or segment mapping must fail CI and must not be deployed. Container images will be tagged with the Git commit SHA. Secrets and credentials will not be committed to the repository.

8. Cost Estimate

The dataset and model are small, so training and inference require only CPU resources. We will use a small serverless container that can scale to zero.

Our estimated project cost is **below 150 THB**, excluding any free credits. We will set a cloud budget alert, record actual cloud billing, calculate the cost per 1,000 predictions, and remove all running resources after testing.

9. Team Responsibilities

- **[Member 1]:** Dataset, DVC, preprocessing, training pipeline, MLflow, and model registry
- **[Member 2]:** FastAPI, Docker, cloud deployment, CI/CD, and load testing
- **[Member 3]:** Dashboard, drift detection, alert, failure demonstration, cost report, and model card

All members will review the cluster names, tests, documentation, final repository, and live demonstration.